

UNIVERSITY OF ASIA PACIFIC



Course Code: CSE 430

Course Title: Compiler Lab

**LexiCore: An Interactive GUI-Based Mini C Compiler A Complete
Implementation of Six-Phase Compiler Pipeline with LL(1) Grammar
Transformation**

Prepared for:
Md. Shafiqul Islam, Ph.D.
Ass.Professor , Dept. of CSE
University of Asia Pacific

Prepared by:
Abdullah Al-Mamun
ID No: 22101158
Sec: D

**Department of Computer Science and Engineering
University of Asia Pacific**

Table of Contents

ABSTRACT.....	3
1. System Overview.....	3
1.1 Technology Stack	4
2. Compilation Pipeline — Block Diagram.....	4
3. LL(1) Grammar Transformation Flow	6
4. Module Interaction Diagram.....	8
6. Project File Structure	9
7.1 Lexical Analyzer (lexer.py)	10
7.2 Syntax Processor (parser.py)	15
7.3 Symbol Table (symbol_table.py).....	36
7.4 Semantic Analyzer (semantic.py).....	39
7.5 Code Generator (code_generator.py).....	42
7.6 Grammar Utilities (grammar_utils.py)	49
7.6.1 Grammar Normalizer.....	49
7.6.2 Left Recursion Removal	50
7.6.3 Left Factoring	52
7.6.3 First () & Follow ()	54
8. Test Cases & Expected Output	57
8.1 Test Case 1: Prime Number	57
9. Phase Summary.....	64
10. References.....	65

ABSTRACT

LexiCore is a fully functional, GUI-driven Mini C Compiler developed entirely in Python, designed to demonstrate the complete workflow of modern compiler construction in an interactive and educational way. The system processes C-like source code through six major compilation phases: Lexical Analysis, Syntax Analysis, Symbol Table Management, Semantic Analysis, Intermediate Representation (IR/TAC) Generation, and x86 Assembly Code Generation.

The compiler is built using PLY (Python Lex-Yacc) with an LALR(1)-based parser and features a modern dark-themed IDE-style graphical interface developed with Tkinter. Each compilation phase is visualized in real time through dedicated GUI tabs, allowing users to clearly observe token generation, syntax parsing, semantic checking, intermediate code creation, and final assembly output.

LexiCore also includes an advanced LL(1) Grammar Transformation module capable of performing left recursion elimination, left factoring, and FIRST/FOLLOW set computation for grammar preprocessing and parser analysis. Additional features such as scope-aware symbol table handling, type checking, backpatching, temporary variable management, and assembly-level code generation make the project a comprehensive demonstration of compiler design concepts.

The project serves both as an educational learning platform and as a practical implementation of core compiler engineering principles.

1. System Overview

LexiCore is a complete, multi-phase Mini C Compiler designed to demonstrate the full workflow of compiler construction in an interactive and user-friendly environment. The system accepts C-like source code through a modern IDE-style graphical interface and processes it through six sequential compilation phases: Lexical Analysis, Syntax Analysis, Symbol Table Construction, Semantic Analysis, Intermediate Representation (IR/TAC) Generation, and x86 Assembly Code Generation.

Each phase is implemented as an independent module, allowing clear separation of responsibilities and easier debugging, maintenance, and visualization. As the source code moves through the compiler pipeline, the output of every phase is displayed in dedicated GUI tabs, enabling users to observe how raw source code is gradually transformed into low-level assembly instructions.

The compiler is developed entirely in Python using PLY (Python Lex-Yacc) for lexical and syntax analysis, while Tkinter is used to build the dark-themed graphical interface. The architecture follows a modular pipeline design where each phase passes processed data to the next stage in sequence.

In addition to the main compiler pipeline, LexiCore also includes a standalone LL(1) Grammar Transformation module accessible from the main toolbar. This tool supports grammar

preprocessing operations such as left recursion elimination, left factoring, and FIRST/FOLLOW set computation, helping users understand fundamental parsing and compiler theory concepts.

1.1 Technology Stack

Language	Python 3.10+
Lexer / Parser	PLY (Python Lex-Yacc) — LALR(1)
GUI Framework	Tkinter + ttk (dark Dracula theme)
IR	Three-Address Code (TAC) — dictionary-based
Code Generator	Pseudo x86 Assembly (NASM-compatible output)
LL(1) Analysis	Custom recursive-descent grammar_utils module

2. Compilation Pipeline — Block Diagram

Whenever the user clicks the **Run** button in the LexiCore IDE, the source code begins its journey through a structured six-phase compilation pipeline. Each phase performs a specific task in the compilation process and passes its processed output directly to the next phase. This sequential workflow allows the compiler to gradually transform high-level C-like source code into low-level assembly instructions.

The pipeline starts with **Lexical Analysis**, where the source code is scanned character by character and converted into tokens such as keywords, identifiers, literals, and operators. These tokens are then forwarded to the **Syntax Analysis (Parser)** phase, where grammar rules are applied to validate the program structure and generate an Abstract Syntax Tree (AST).

Next, the compiler enters the **Symbol Table Construction** phase, where variables, functions, scopes, and memory-related information are stored and managed. After that, the **Semantic Analysis** phase verifies logical correctness by detecting undeclared identifiers, redeclarations, type mismatches, and invalid function calls.

Once semantic validation is complete, the compiler generates **Intermediate Representation (IR)** in the form of Three Address Code (TAC). This intermediate code simplifies complex expressions into smaller low-level operations using temporaries and labels. Finally, the IR is translated into **x86 Assembly Code**, producing pseudo-assembly instructions that represent the final low-level output of the compiler.

This modular pipeline architecture makes the compiler easier to understand, debug, maintain, and extend, while also providing users with a clear visualization of how source code is transformed step by step inside a real compiler system.

LexiCore Compilation Pipeline

SOURCE CODE (C-like input in the editor)

int, float, void · if/else · while · functions · print



Phase 1 — Lexical Analysis

lexer.py → TokenScanner (PLY Lex)

Input: raw source string | Output: token stream [{ kind, val, ln, pos }]

Rules: keywords, identifiers, numbers, operators, comments stripped, line numbers tracked



Phase 2 — Syntax Analysis

parser.py → SyntaxProcessor (PLY LALR(1)) + syntax_analysis.py

Input: token stream | Output: AST (nested tuples) + grammar productions fired

Handles: var_decl, assignments, if/else, while, functions, return, expressions



Phase 3 — Symbol Table Construction

symbol_table.py → VariableRegistry

Input: AST traversal during parse | Output: scoped symbol dictionary

Features: nested scope stack, shadow detection, address assignment, function/param tracking



Phase 4 — Semantic Analysis

semantic.py → semantic_analysis()

Input: AST + symbol data | Output: list of semantic errors

Checks: undeclared identifiers, redeclaration, function arity, return-type compatibility, type widening



Phase 5 — IR / TAC Generation

parser.py (emit() calls) → ir_instructions list

Input: AST reductions | Output: list of { op, src1, src2, dst } dicts

Ops: assign, +/-/*/, %, </>/ == / !=, jump, jump_if_false, mark, output, call, return



Phase 6 — Assembly Code Generation

code_generator.py → AssemblyTranslator

Input: IR instruction list | Output: NASM-compatible x86 assembly lines

Features: register allocation (round-robin), CDQ/IDIV for division/modulo, SETcc for comparisons

3. LL(1) Grammar Transformation Flow

The LL(1) Grammar Transformation Tool is a standalone educational module integrated into the LexiCore IDE and accessible directly from the top toolbar. This module is designed to help users understand important concepts of compiler theory and predictive parsing by allowing them to input and analyze custom context-free grammars interactively.

When a grammar is entered, the tool automatically applies several classical LL(1) preprocessing techniques required for constructing predictive parsers. One of the primary operations performed is **Left Recursion Elimination**, where directly or indirectly recursive productions are transformed into right-recursive forms to prevent infinite recursion during top-down parsing. This conversion is essential for making grammars compatible with recursive-descent and LL(1) parsing methods.

The tool also performs **Left Factoring**, which identifies common prefixes among productions and restructures them into a factored form. This transformation removes ambiguity in parser decision-making by ensuring that each production can be selected using a single lookahead token. As a result, the grammar becomes more suitable for deterministic predictive parsing.

In addition to grammar transformation, the module computes **FIRST** and **FOLLOW** sets for all non-terminals. The FIRST set determines the terminals that may appear at the beginning of a derivation, while the FOLLOW set identifies which terminals may legally appear after a non-terminal in a valid sentence. These sets are fundamental components used in LL(1) parsing table construction and syntax analysis.

The entire transformation process is displayed step by step in a structured and easy-to-understand format. Users can observe the original grammar, transformed productions, and generated FIRST/FOLLOW sets directly inside the GUI. This visual and interactive approach makes the module highly useful for learning grammar simplification, parser preparation, ambiguity resolution, and core compiler design principles.

By integrating grammar preprocessing tools into the compiler environment, LexiCore not only functions as a compiler implementation project but also serves as an educational platform for understanding formal language processing and parsing theory in practice.

LL(1) Grammar Transformer Flow

Step 1: User Types Grammar

$S \rightarrow iEtS \mid a$
 $A \rightarrow BD \mid h$
(compact or spaced notation)



Step 2: Tokenize Symbols

`_normalize_grammar()` splits each production into atomic symbols
NTs kept whole · ϵ kept · known terminals (id, num, if, ...) kept · rest split char-by-char



Step 3: Remove Left Recursion

`remove_left_recursion()`
Direct and indirect recursion eliminated via ordered NT substitution
New prime symbols (A') created as needed



Step 4: Left Factoring

`left_factoring()`
Productions grouped by first symbol · longest common prefix extracted
Loop runs until no further factoring is possible



Step 5: FIRST / FOLLOW Sets

`compute_first()` + `compute_follow()`
Fixed-point iteration until no changes
Follow seeds start set with $\$$



Step 6: Display Results

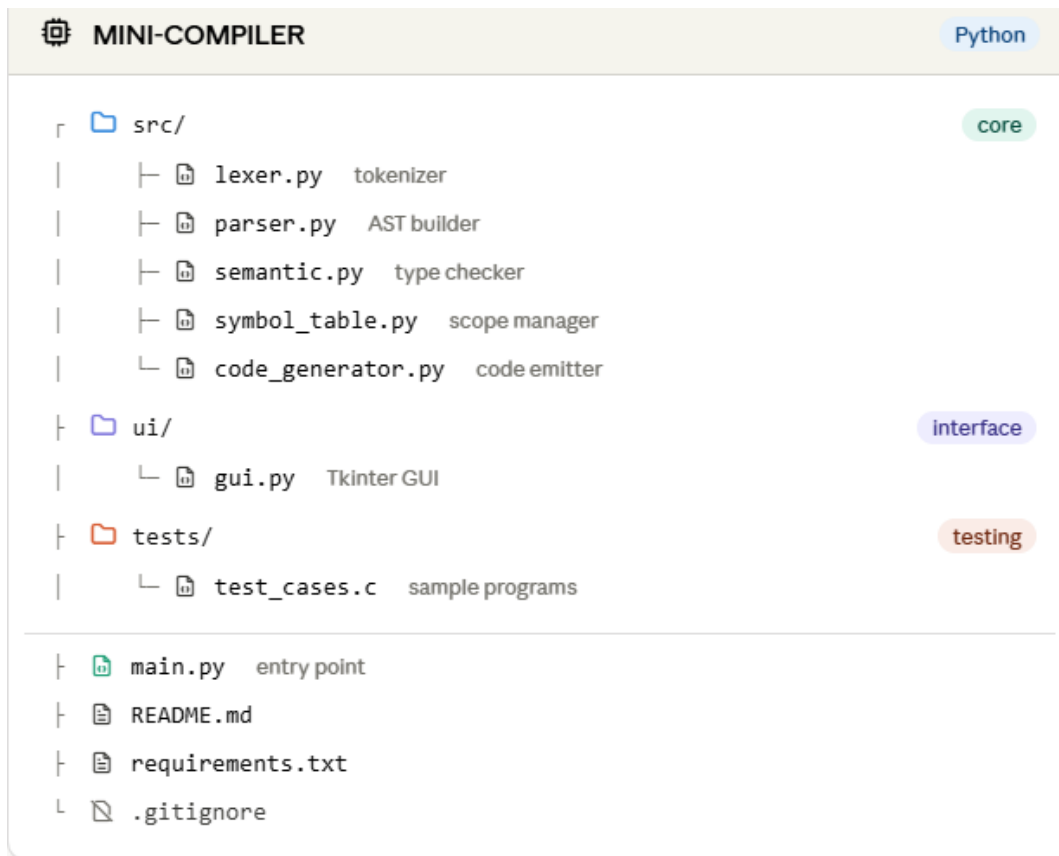
Coloured output panel with LL(1) syntax highlighting
NT symbols, arrows, epsilon, and prime names each have distinct colours

4. Module Interaction Diagram

The Module Interaction Diagram shows how different Python modules in LexiCore communicate with each other during compilation. Arrows in the diagram represent import and function-call relationships between modules.

The `gui.py` module acts as the central controller, managing user input and coordinating all compiler phases. The `lexer.py` module generates tokens, which are passed to `parser.py` for syntax analysis and AST generation. The `symbol_table.py` and `semantic.py` modules handle scope management, type checking, and semantic validation.

After successful analysis, `tac.py` generates Three Address Code (TAC), and `asm.py` converts the intermediate code into x86 assembly instructions. Separate modules such as `grammar_preprocess.py`, `grammar_tool.py`, and `grammar_utils.py` manage LL(1) grammar transformations and FIRST/FOLLOW computations independently from the main compiler pipeline.



6. Project File Structure

The following table presents the core modules of the **Mini Compiler** project along with their main responsibilities in a structured and organized manner. Each module is carefully designed to implement a specific phase of the compiler pipeline, ensuring modularity, readability, and ease of maintenance. These phases include lexical analysis, syntax parsing, semantic analysis, symbol table management, intermediate and target code generation, as well as grammar transformation utilities such as left recursion elimination and left factoring.

By dividing the compiler into separate functional modules, the project achieves a clean architecture where each component operates independently but collaboratively contributes to the overall compilation process. This modular approach not only improves debugging and testing efficiency but also makes the system more scalable and easier to extend with additional language features in the future.

FILE	MODULE	RESPONSIBILITY
☐ <code>gui.py</code>	<code>CompilerInterface</code>	Main Tkinter window: editor, output tabs, snippet bar, compilation orchestration
☐ <code>lexer.py</code>	<code>TokenScanner</code>	PLY-based lexer: keywords, identifiers, numbers, operators, single/multi-line comments
☐ <code>parser.py</code>	<code>SyntaxProcessor</code>	PLY LALR(1) parser + inline IR emitter: grammar rules, scoping, error recovery
☐ <code>syntax_analysis.py</code>	<code>run_syntax_analysis</code>	Wrapper that patches the parser to collect fired productions and formats the AST
☐ <code>symbol_table.py</code>	<code>VariableRegistry</code>	Nested-scope symbol table: push/pop, shadow detection, declaration order tracking
☐ <code>semantic.py</code>	<code>semantic_analysis</code>	Semantic checks: undeclared use, redeclaration, arity mismatch, return type compatibility
☐ <code>code_generator.py</code>	<code>AssemblyTranslator</code>	IR-to-x86 translator: register allocation, CDQ/IDIV, SETcc, function prologue/epilogue
☐ <code>grammar_utils.py</code>	<code>grammar_utils</code>	LL(1) library: normalizer, left-recursion removal, left-factoring, FIRST/FOLLOW
☐ <code>grammar_tool.py</code>	<code>CLI tool</code>	Command-line front-end for <code>grammar_utils</code> ; used for standalone grammar testing
☐ <code>parsetab.py</code>	<code>PLY generated</code>	Auto-generated LALR parse table — not edited manually
☐ <code>__init__.py</code>	<code>Package</code>	Empty package marker

7. Source Code Listings

7.1 Lexical Analyzer (lexer.py)

The TokenScanner class uses PLY's lex module. It defines regex-based token rules for all terminal symbols and discards whitespace and comments.

Key Design Points

- Two-character operators (==, !=, <=, >=) are declared as string attributes (higher PLY priority) so they are matched before single-char operators.
- Multi-line comments use a token function rather than a string rule so the line counter can be updated on each embedded newline.
- The keyword map is checked inside t_IDENTIFIER so keyword priority is handled without separate rules.
- Each token dict returned by scan() carries: kind, val, ln (line number), pos (character position).

```
import ply.lex as lex

class TokenScanner:
    """Lexical analyzer – supports both LexiCore syntax and C-
    style code."""

    keywords = {
        'if': 'IF',
        'else': 'ELSE',
        'while': 'WHILE',
        'for': 'FOR',
        'int': 'INT',
        'float': 'FLOAT',
        'void': 'VOID',
        'return': 'RETURN',
        'print': 'PRINT',
        'scanf': 'SCANF',
        'printf': 'PRINTF',
        'break': 'BREAK',
        'continue': 'CONTINUE',
        'char': 'CHAR',
        'double': 'DOUBLE',
        'long': 'LONG',
```

```

        'short': 'SHORT',
        'unsigned': 'UNSIGNED',
        'main': 'MAIN',
    }

    tokens = [
        'IDENTIFIER', 'INTEGER', 'DECIMAL',
        'STRING_LIT',
        'PLUS', 'MINUS', 'MULTIPLY', 'DIVIDE', 'MOD',
        'EQUALS',
        'EQUAL_TO', 'NOT_EQUAL',
        'LESS_EQ', 'GREATER_EQ', 'LESS', 'GREATER',
        'LPAREN', 'RPAREN', 'LBRACE', 'RBRACE',
        'LBRACKET', 'RBRACKET',
        'SEMICOLON', 'COMMA',
        'AND', 'OR', 'NOT',
        'INCREMENT', 'DECREMENT',
        'PLUS_ASSIGN', 'MINUS_ASSIGN', 'MUL_ASSIGN',
'DIV_ASSIGN',
        'AMPERSAND',
        'HASH',
        'DOT',
    ] + list(keywords.values())

    # — Preprocessor lines (#include, #define) – skip entirely


---


    def t_PREPROCESSOR(self, tok):
        r'\#[^\n]*'
        pass # discard preprocessor directives

    # — Single-line comment


---


    def t_COMMENT_SINGLE(self, tok):
        r'//[^\n]*'
        pass

    # — Multi-line comment


---


    def t_COMMENT_MULTI(self, tok):

```

```

    r'\/\*(.|\n)*?\*/'
    tok.lexer.lineno += tok.value.count('\n')
    pass

# — String literals


---


def t_STRING_LIT(self, tok):
    r'"([^"\\]|\\.)*"'
    return tok

# — Three / two-char operators (must appear BEFORE shorter
ones)


---


t_AND      = r'&&'
t_OR       = r'\|\|'
t_INCREMENT = r'\++'
t_DECREMENT = r'--'
t_PLUS_ASSIGN = r'\+='
t_MINUS_ASSIGN = r'-='
t_MUL_ASSIGN = r'\*='
t_DIV_ASSIGN = r'/='
t_EQUAL_TO   = r'=='
t_NOT_EQUAL  = r'!='
t_LESS_EQ    = r'<='
t_GREATER_EQ = r'>='

# — Single-char operators


---


t_PLUS      = r'\+'
t_MINUS     = r'\-'
t_MULTIPLY  = r'\*'
t_DIVIDE    = r'\/'
t_MOD       = r'\%'
t_EQUALS    = r'='
t_LESS      = r'<'
t_GREATER   = r'>'
t_NOT       = r'!'
t_AMPERSAND = r'&'
t_DOT       = r'\.'

```

— Delimiters

```
t_LPAREN      = r'\('
t_RPAREN      = r'\)'
t_LBRACE      = r'\{'
t_RBRACE      = r'\}'
t_LBRACKET    = r'\['
t_RBRACKET    = r'\]'
t_SEMICOLON   = r';'
t_COMMA       = r','
t_HASH        = r'\#'
```

```
t_ignore = ' \t'
```

— Literals

```
def t_DECIMAL(self, tok):
    r'\d+\.\d+'
    tok.value = float(tok.value)
    return tok

def t_INTEGER(self, tok):
    r'\d+'
    tok.value = int(tok.value)
    return tok

def t_IDENTIFIER(self, tok):
    r'[a-zA-Z_][a-zA-Z_0-9]*'
    tok.type = self.keywords.get(tok.value, 'IDENTIFIER')
    return tok

def t_newline(self, tok):
    r'\n+'
    tok.lexer.lineno += len(tok.value)

def t_error(self, tok):
    self.issues.append(
        f"Line {tok.lineno}: Invalid character
'{tok.value[0]}'")
```

```

    )
    tok.lexer.skip(1)

# — Public API


---


def __init__(self):
    self.scanner = None
    self.token_stream = []
    self.issues = []

def initialize(self):
    self.scanner = lex.lex(module=self)

def scan(self, code: str):
    self.token_stream = []
    self.issues = []
    self.scanner.lineno = 1
    self.scanner.input(code)

    while True:
        tok = self.scanner.token()
        if not tok:
            break
        self.token_stream.append({
            'kind': tok.type,
            'val': tok.value,
            'ln': tok.lineno,
            'pos': tok.lexpos,
        })

    return self.token_stream, self.issues

```

7.2 Syntax Processor (parser.py)

The `SyntaxProcessor` class embeds PLY grammar rules as `p_XXX` methods. Each rule simultaneously builds the AST and emits three-address IR instructions.

Key Design Points

- Operator precedence: PLUS/MINUS < MULTIPLY/DIVIDE/MOD, left-associative, declared via the precedence tuple.
- Scoping is managed by calling `self.registry.push_scope()` on LBRACE and `pop_scope()` on RBRACE via mid-rule actions (`p_block_start`, `p_block_end`).
- Loop and conditional back-patching uses stacks (`_if_false_stack`, `_while_start_stack`, etc.) so nested structures work correctly.
- Error recovery skips tokens until SEMICOLON or RBRACE to allow continued parsing after syntax errors.
- Function declarations emit `func_begin`, `param`, `return` instructions; calls emit `arg` then `call` instructions.

```
import ply.yacc as yacc
from lexer import TokenScanner
from symbol_table import VariableRegistry

class SyntaxProcessor:
    """
    LALR(1) parser + IR emitter.
    Supports LexiCore syntax AND C-style code.
    Fixes: comma-separated declarations, braceless
    if/else/for/while,
           break inside loops, return 0, multi-var
    declarations.
    """

    tokens = TokenScanner.tokens

    precedence = (
        ('left', 'OR'),
        ('left', 'AND'),
        ('right', 'NOT'),
        ('left', 'EQUAL_TO', 'NOT_EQUAL'),
        ('left', 'LESS', 'LESS_EQ', 'GREATER', 'GREATER_EQ'),
        ('left', 'PLUS', 'MINUS'),
```

```

        ('left', 'MULTIPLY', 'DIVIDE', 'MOD'),
        ('right', 'UMINUS'),
        ('left', 'INCREMENT', 'DECREMENT'),
    )

    def __init__(self):
        self.registry = VariableRegistry()
        self.ir_instructions = []
        self.tmp_counter = 0
        self.lbl_counter = 0
        self.issues = []
        self.ast = []
        self._last_cond = None

    def gen_temp(self) -> str:
        self.tmp_counter += 1
        return f"t{self.tmp_counter}"

    def gen_label(self) -> str:
        self.lbl_counter += 1
        return f"L{self.lbl_counter}"

    def emit(self, op, src1=None, src2=None, dst=None):
        self.ir_instructions.append(
            {'op': op, 'src1': src1, 'src2': src2, 'dst': dst}
        )
        return dst

#
=====
# TOP LEVEL
#
=====

def p_start(self, p):
    'start : stmt_sequence'
    p[0] = ('program', p[1])

```

```

        self.ast.append(p[0])

def p_stmt_sequence(self, p):
    '''stmt_sequence : stmt_sequence stmt
                      | stmt'''
    p[0] = p[1] + [p[2]] if len(p) == 3 else [p[1]]

def p_stmt(self, p):
    '''stmt : var_decl
            | var_decl_multi
            | var_assign
            | compound_assign
            | output_stmt
            | input_stmt
            | conditional
            | loop
            | for_loop
            | code_block
            | func_decl
            | return_stmt
            | break_stmt
            | continue_stmt
            | expr_stmt
            | increment_stmt'''
    p[0] = p[1]

# — Data types

```

```

def p_data_type(self, p):
    '''data_type : INT
                 | FLOAT
                 | VOID
                 | CHAR
                 | DOUBLE
                 | LONG
                 | SHORT
                 | UNSIGNED'''
    p[0] = p[1]

```

— Single variable declaration

```
def p_var_decl(self, p):
    '''var_decl : data_type IDENTIFIER SEMICOLON
                | data_type IDENTIFIER EQUALS expr
SEMICOLON'''
    dtype, name = p[1], p[2]
    if self.registry.is_declared_in_current_scope(name):
        self.issues.append(f"Error: '{name}' already
declared in this scope")
    else:
        if len(p) == 4:
            self.registry.add(name, dtype, None)
            p[0] = ('decl', dtype, name)
        else:
            val = p[4]
            self.registry.add(name, dtype, val)
            self.emit('assign', val, None, name)
            p[0] = ('decl_init', dtype, name, val)
```

— Comma-separated declaration: int n, i, flag = 0;

```
def p_var_decl_multi(self, p):
    'var_decl_multi : data_type decl_list SEMICOLON'
    p[0] = ('decl_multi', p[1], p[2])

def p_decl_list(self, p):
    '''decl_list : decl_list COMMA decl_item
                | decl_item'''
    if len(p) == 4:
        p[0] = p[1] + [p[3]]
    else:
        p[0] = [p[1]]

def p_decl_item(self, p):
    '''decl_item : IDENTIFIER
```

```

        | IDENTIFIER EQUALS expr'''
dtype = getattr(self, '_current_decl_type', 'int')
name = p[1]
if self.registry.is_declared_in_current_scope(name):
    self.issues.append(f"Error: '{name}' already
declared in this scope")
else:
    if len(p) == 2:
        self.registry.add(name, dtype, None)
        p[0] = ('decl', dtype, name)
    else:
        val = p[3]
        self.registry.add(name, dtype, val)
        self.emit('assign', val, None, name)
        p[0] = ('decl_init', dtype, name, val)

```

Inject type into decl_item via p_var_decl_multi hook

```

def p_decl_type_inject(self, p):
    'decl_type_inject : data_type'
    self._current_decl_type = p[1]
    p[0] = p[1]

```

— Function Declaration

```

def p_func_decl(self, p):
    '''func_decl : data_type IDENTIFIER LPAREN param_list
RPAREN code_block
                | data_type IDENTIFIER LPAREN RPAREN
code_block
                | data_type MAIN LPAREN RPAREN code_block
                | data_type MAIN LPAREN param_list RPAREN
code_block'''
    fname = p[2]
    dtype = p[1]
    if len(p) == 7:
        params = p[4]
        body = p[6]
    else:

```

```

        params = []
        body    = p[5]
    if self.registry.is_declared_in_current_scope(fname):
        self.issues.append(f"Error: function '{fname}'
already declared")
    else:
        self.registry.add(fname, dtype, None, 'function')
        self.emit('func_begin', fname, dtype)
        p[0] = ('func_decl', dtype, fname, params, body)

def p_param_list(self, p):
    '''param_list : param_list COMMA param
                  | param'''
    if len(p) == 4:
        p[0] = p[1] + [p[3]]
    else:
        p[0] = [p[1]]

def p_param(self, p):
    '''param : data_type IDENTIFIER'''
    self.registry.add(p[2], p[1], None, 'param')
    self.emit('param', p[2], p[1])
    p[0] = (p[1], p[2])

# — Return / break / continue

def p_return_stmt(self, p):
    '''return_stmt : RETURN expr SEMICOLON
                  | RETURN SEMICOLON'''
    if len(p) == 4:
        self.emit('return', p[2])
        p[0] = ('return', p[2])
    else:
        self.emit('return', None)
        p[0] = ('return', None)

def p_break_stmt(self, p):
    'break_stmt : BREAK SEMICOLON'

```

```

        # Emit jump to nearest loop-end label
        if hasattr(self, '_loop_end_stack') and
self._loop_end_stack:
            self.emit('jump', self._loop_end_stack[-1])
        else:
            self.emit('break', None)
        p[0] = ('break',)

    def p_continue_stmt(self, p):
        'continue_stmt : CONTINUE SEMICOLON'
        if hasattr(self, '_loop_start_stack') and
self._loop_start_stack:
            self.emit('jump', self._loop_start_stack[-1])
        else:
            self.emit('continue', None)
        p[0] = ('continue',)

# — Expression statement

def p_expr_stmt(self, p):
    'expr_stmt : IDENTIFIER LPAREN arg_list RPAREN
SEMICOLON'
    fname = p[1]
    args = p[3]
    if not self.registry.find(fname):
        self.issues.append(f"Error: Undefined function
'{fname}'")
    tmp = self.gen_temp()
    for arg in args:
        self.emit('arg', arg)
    self.emit('call', fname, len(args), tmp)
    p[0] = ('call_stmt', fname, args)

def p_increment_stmt(self, p):
    '''increment_stmt : IDENTIFIER INCREMENT SEMICOLON
                        | IDENTIFIER DECREMENT SEMICOLON'''
    name = p[1]
    op = '+' if p[2] == '++' else '-'

```

```

    tmp = self.gen_temp()
    self.emit(op, name, 1, tmp)
    self.emit('assign', tmp, None, name)
    if self.registry.find(name):
        self.registry.update(name, tmp)
    p[0] = ('incr_stmt', name, p[2])

```

```

def p_arg_list_many(self, p):
    'arg_list : arg_list COMMA expr'
    p[0] = p[1] + [p[3]]

```

```

def p_arg_list_one(self, p):
    'arg_list : expr'
    p[0] = [p[1]]

```

```

def p_arg_list_empty(self, p):
    'arg_list :'
    p[0] = []

```

— Assignment

```

def p_var_assign(self, p):
    'var_assign : IDENTIFIER EQUALS expr SEMICOLON'
    name, val = p[1], p[3]
    if not self.registry.find(name):
        self.issues.append(f"Error: Undefined variable '{name}'")
    else:
        self.registry.update(name, val)
        self.emit('assign', val, None, name)
    p[0] = ('assign', name, val)

```

```

def p_compound_assign(self, p):
    '''compound_assign : IDENTIFIER PLUS_ASSIGN expr
    SEMICOLON
                        | IDENTIFIER MINUS_ASSIGN expr
    SEMICOLON

```

```

| IDENTIFIER MUL_ASSIGN expr
SEMICOLON
| IDENTIFIER DIV_ASSIGN expr
SEMICOLON'''
    name = p[1]
    op_map = {'+=': '+', '-=': '-', '*=': '*', '/=': '/'}
    op = op_map[p[2]]
    tmp = self.gen_temp()
    self.emit(op, name, p[3], tmp)
    self.emit('assign', tmp, None, name)
    if self.registry.find(name):
        self.registry.update(name, tmp)
    p[0] = ('compound_assign', name, p[2], p[3])

```

— Output

```

def p_output_stmt(self, p):
    'output_stmt : PRINT LPAREN expr RPAREN SEMICOLON'
    self.emit('output', p[3])
    p[0] = ('output', p[3])

def p_printf_stmt(self, p):
    '''output_stmt : PRINTF LPAREN STRING_LIT RPAREN
SEMICOLON
| PRINTF LPAREN STRING_LIT COMMA
printf_args RPAREN SEMICOLON'''
    if len(p) == 6:
        self.emit('output', p[3])
        p[0] = ('printf', p[3])
    else:
        for arg in p[5]:
            self.emit('output', arg)
        p[0] = ('printf', p[3], p[5])

def p_printf_args(self, p):
    '''printf_args : printf_args COMMA expr
| expr'''
    if len(p) == 4:

```

```

        p[0] = p[1] + [p[3]]
    else:
        p[0] = [p[1]]

# — Input (scanf)



---



def p_input_stmt(self, p):
    '''input_stmt : SCANF LPAREN STRING_LIT COMMA AMPERSAND
IDENTIFIER RPAREN SEMICOLON
                  | SCANF LPAREN STRING_LIT COMMA
scanf_args RPAREN SEMICOLON'''
    if len(p) == 9:
        name = p[6]
        self.emit('input', name, None, name)
        if not self.registry.find(name):
            self.issues.append(f"Warning: '{name}' used in
scanf but not declared")
        p[0] = ('scanf', name)
    else:
        p[0] = ('scanf_multi', p[5])

def p_scanf_args(self, p):
    '''scanf_args : scanf_args COMMA AMPERSAND IDENTIFIER
                  | AMPERSAND IDENTIFIER'''
    if len(p) == 5:
        name = p[4]
        self.emit('input', name, None, name)
        p[0] = p[1] + [name]
    else:
        name = p[2]
        self.emit('input', name, None, name)
        p[0] = [name]

#



---




---



# BODY – braced block OR single statement (braceless)

```

```

#
=====

def p_body_block(self, p):
    'body : code_block'
    p[0] = p[1]

def p_body_single(self, p):
    '''body : var_assign
            | compound_assign
            | output_stmt
            | input_stmt
            | return_stmt
            | break_stmt
            | continue_stmt
            | increment_stmt
            | expr_stmt'''
    p[0] = ('single_stmt', p[1])

# — If / If-else (both braced and braceless)

def p_conditional_if(self, p):
    'conditional : IF LPAREN comparison RPAREN if_jump body
if_end'
    p[0] = ('if', p[3], p[6])

def p_conditional_ifelse(self, p):
    'conditional : IF LPAREN comparison RPAREN if_jump body
else_jump ELSE body if_end'
    p[0] = ('ifelse', p[3], p[6], p[9])

def p_if_jump(self, p):
    'if_jump :'
    lbl_false = self.gen_label()
    self.emit('jump_if_false', self._last_cond, lbl_false)
    self._if_false_stack = getattr(self, '_if_false_stack',
[ ])

```

```

        self._if_false_stack.append(lbl_false)
        p[0] = None

def p_else_jump(self, p):
    'else_jump :'
    lbl_end = self.gen_label()
    self._if_end_stack = getattr(self, '_if_end_stack', [])
    self._if_end_stack.append(lbl_end)
    self.emit('jump', lbl_end)
    lbl_false = self._if_false_stack.pop()
    self.emit('mark', lbl_false)
    p[0] = None

def p_if_end(self, p):
    'if_end :'
    if hasattr(self, '_if_end_stack') and
self._if_end_stack:
        lbl_end = self._if_end_stack.pop()
        self.emit('mark', lbl_end)
    else:
        lbl_false = self._if_false_stack.pop()
        self.emit('mark', lbl_false)
    p[0] = None

# — While loop (braced and braceless)

def p_loop(self, p):
    'loop : WHILE while_start LPAREN comparison RPAREN
while_body body while_end'
    p[0] = ('while', p[4], p[7])

def p_while_start(self, p):
    'while_start :'
    lbl_start = self.gen_label()
    self.emit('mark', lbl_start)
    self._while_start_stack = getattr(self,
'_while_start_stack', [])
    self._while_start_stack.append(lbl_start)

```

```

        self._loop_start_stack = getattr(self,
'_loop_start_stack', [])
        self._loop_start_stack.append(lbl_start)
        p[0] = None

```

```

def p_while_body(self, p):
    'while_body :'
    cond = self._last_cond
    lbl_end = self.gen_label()
    self._while_end_stack = getattr(self,
'_while_end_stack', [])
    self._while_end_stack.append(lbl_end)
    self._loop_end_stack = getattr(self,
'_loop_end_stack', [])
    self._loop_end_stack.append(lbl_end)
    self.emit('jump_if_false', cond, lbl_end)
    p[0] = None

```

```

def p_while_end(self, p):
    'while_end :'
    lbl_start = self._while_start_stack.pop()
    lbl_end = self._while_end_stack.pop()
    self._loop_start_stack.pop()
    self._loop_end_stack.pop()
    self.emit('jump', lbl_start)
    self.emit('mark', lbl_end)
    p[0] = None

```

— For loop

```

def p_for_loop(self, p):
    'for_loop : FOR LPAREN for_init SEMICOLON
for_mark_start comparison SEMICOLON for_update RPAREN for_body
body for_end'
    p[0] = ('for', p[6], p[11])

```

```

def p_for_init(self, p):
    '''for_init : data_type IDENTIFIER EQUALS expr

```

```

        | IDENTIFIER EQUALS expr
        | IDENTIFIER INCREMENT
        | IDENTIFIER DECREMENT
        | INCREMENT IDENTIFIER
        | DECREMENT IDENTIFIER
        |'''
    if len(p) == 5:
        dtype, name, val = p[1], p[2], p[4]
        if not
self.registry.is_declared_in_current_scope(name):
            self.registry.add(name, dtype, val)
            self.emit('assign', val, None, name)
            p[0] = ('for_init_decl', dtype, name, val)
    elif len(p) == 4:
        name, val = p[1], p[3]
        self.emit('assign', val, None, name)
        p[0] = ('for_init_assign', name, val)
    elif len(p) == 3:
        if p[1] in ('++', '--'):
            name = p[2]; op = '+' if p[1] == '++' else '-'
        else:
            name = p[1]; op = '+' if p[2] == '++' else '-'
        tmp = self.gen_temp()
        self.emit(op, name, 1, tmp)
        self.emit('assign', tmp, None, name)
        p[0] = ('for_init_incr', name)
    else:
        p[0] = None

def p_for_mark_start(self, p):
    'for_mark_start :'
    lbl = self.gen_label()
    self.emit('mark', lbl)
    self._for_start_stack = getattr(self,
'_for_start_stack', [])
    self._for_start_stack.append(lbl)
    self._loop_start_stack = getattr(self,
'_loop_start_stack', [])
    self._loop_start_stack.append(lbl)

```

```

        p[0] = None

def p_for_update(self, p):
    '''for_update : IDENTIFIER EQUALS expr
                  | IDENTIFIER INCREMENT
                  | IDENTIFIER DECREMENT
                  | INCREMENT IDENTIFIER
                  | DECREMENT IDENTIFIER
                  | IDENTIFIER PLUS_ASSIGN expr
                  | IDENTIFIER MINUS_ASSIGN expr
                  | IDENTIFIER MUL_ASSIGN expr
                  | IDENTIFIER DIV_ASSIGN expr
                  |'''

    self._for_update_instrs = []
    if len(p) == 4 and p[2] == '=':
        self._for_update_instrs = [('assign', p[3], None,
p[1])]

        p[0] = ('for_update_assign', p[1], p[3])
    elif len(p) == 3:
        if p[1] in ('++', '--'):
            name = p[2]; op = '+' if p[1] == '++' else '-'
        else:
            name = p[1]; op = '+' if p[2] == '++' else '-'
            tmp = f"_fu{self.tmp_counter+1}"
            self._for_update_instrs = [(op, name, 1, tmp),
('assign', tmp, None, name)]
            p[0] = ('for_update_incr', name)
    elif len(p) == 4:
        op_map = {'+=': '+', '--': '-', '*=': '*', '/=':
'/'}

        op = op_map.get(p[2], '+')
        tmp = f"_fu{self.tmp_counter+1}"
        self._for_update_instrs = [(op, p[1], p[3], tmp),
('assign', tmp, None, p[1])]
        p[0] = ('for_update_compound', p[1], p[2], p[3])
    else:
        p[0] = None

def p_for_body(self, p):

```

```

        'for_body :'
        cond      = self._last_cond
        lbl_end   = self.gen_label()
        self._for_end_stack = getattr(self, '_for_end_stack',
[ ])
        self._for_end_stack.append(lbl_end)
        self._loop_end_stack = getattr(self, '_loop_end_stack',
[ ])
        self._loop_end_stack.append(lbl_end)
        self.emit('jump_if_false', cond, lbl_end)
        p[0] = None

def p_for_end(self, p):
    'for_end :'
    for instr in getattr(self, '_for_update_instrs', []):
        self.ir_instructions.append(
            {'op': instr[0], 'src1': instr[1], 'src2':
instr[2], 'dst': instr[3]}
        )
    lbl_start = self._for_start_stack.pop()
    lbl_end   = self._for_end_stack.pop()
    self._loop_start_stack.pop()
    self._loop_end_stack.pop()
    self.emit('jump', lbl_start)
    self.emit('mark', lbl_end)
    p[0] = None

# — Blocks

```

```

def p_code_block(self, p):
    'code_block : block_start stmt_sequence block_end'
    p[0] = ('block', p[2])

def p_block_start(self, p):
    'block_start : LBRACE'
    self.registry.push_scope()
    p[0] = 'block_start'

```

```
def p_block_end(self, p):
    'block_end : RBRACE'
    self.registry.pop_scope()
    p[0] = 'block_end'
```

— Comparisons

```
def p_comparison(self, p):
    'comparison : expr rel_op expr'
    tmp = self.gen_temp()
    self.emit(p[2], p[1], p[3], tmp)
    self._last_cond = tmp
    p[0] = tmp
```

```
def p_comparison_expr(self, p):
    'comparison : expr'
    self._last_cond = p[1]
    p[0] = p[1]
```

```
def p_rel_op(self, p):
    '''rel_op : LESS
              | LESS_EQ
              | GREATER
              | GREATER_EQ
              | EQUAL_TO
              | NOT_EQUAL'''
    p[0] = p[1]
```

— Expressions

```
def p_expr_or(self, p):
    'expr : expr OR expr'
    tmp = self.gen_temp()
    self.emit('||', p[1], p[3], tmp)
    p[0] = tmp
```

```
def p_expr_and(self, p):
```

```

    'expr : expr AND expr'
    tmp = self.gen_temp()
    self.emit('&&', p[1], p[3], tmp)
    p[0] = tmp

def p_expr_not(self, p):
    'expr : NOT expr'
    tmp = self.gen_temp()
    self.emit('!', p[2], None, tmp)
    p[0] = tmp

def p_expr_add(self, p):
    '''expr : expr PLUS term
        | expr MINUS term'''
    tmp = self.gen_temp()
    self.emit(p[2], p[1], p[3], tmp)
    p[0] = tmp

def p_expr_cmp(self, p):
    '''expr : expr EQUAL_TO expr
        | expr NOT_EQUAL expr
        | expr LESS expr
        | expr LESS_EQ expr
        | expr GREATER expr
        | expr GREATER_EQ expr'''
    tmp = self.gen_temp()
    self.emit(p[2], p[1], p[3], tmp)
    p[0] = tmp

def p_expr_term(self, p):
    'expr : term'
    p[0] = p[1]

def p_term_mul(self, p):
    '''term : term MULTIPLY base
        | term DIVIDE base
        | term MOD base'''
    tmp = self.gen_temp()
    self.emit(p[2], p[1], p[3], tmp)

```

```

        p[0] = tmp

    def p_term_base(self, p):
        'term : base'
        p[0] = p[1]

    def p_base_num(self, p):
        '''base : INTEGER
            | DECIMAL'''
        p[0] = p[1]

    def p_base_string(self, p):
        'base : STRING_LIT'
        p[0] = p[1]

    def p_base_uminus(self, p):
        'base : MINUS base %prec UMINUS'
        tmp = self.gen_temp()
        self.emit('*', p[2], -1, tmp)
        p[0] = tmp

    def p_base_id(self, p):
        'base : IDENTIFIER'
        if not self.registry.find(p[1]):
            self.issues.append(f"Error: Undefined variable
'{p[1]}'")
        p[0] = p[1]

    def p_base_increment(self, p):
        '''base : IDENTIFIER INCREMENT
            | IDENTIFIER DECREMENT'''
        name = p[1]
        op = '+' if p[2] == '++' else '-'
        tmp = self.gen_temp()
        self.emit(op, name, 1, tmp)
        self.emit('assign', tmp, None, name)
        if self.registry.find(name):
            self.registry.update(name, tmp)
        p[0] = name

```

```

def p_base_call(self, p):
    'base : IDENTIFIER LPAREN arg_list RPAREN'
    fname = p[1]
    args = p[3]
    if not self.registry.find(fname):
        self.issues.append(f"Error: Undefined function
'{fname}'")
    for arg in args:
        self.emit('arg', arg)
    tmp = self.gen_temp()
    self.emit('call', fname, len(args), tmp)
    p[0] = tmp

def p_base_paren(self, p):
    'base : LPAREN expr RPAREN'
    p[0] = p[2]

# — Error recovery

def p_error(self, p):
    if p:
        self.issues.append(
            f"Syntax error near '{p.value}' (line
{p.lineno})"
        )
        while True:
            tok = self.processor.token()
            if tok is None or tok.type in ('SEMICOLON',
'RBRACE'):
                break
            self.processor.restart()
    else:
        self.issues.append("Syntax error: unexpected end of
input")

# — Lifecycle

```

```

def initialize(self):
    import logging
    logging.getLogger('ply').setLevel(logging.CRITICAL)
    self.processor = yacc.yacc(
        module=self,
        debug=False,
        write_tables=False,
        errorlog=yacc.NullLogger(),
    )

def process(self, code: str):
    self.ir_instructions = []
    self.tmp_counter = 0
    self.lbl_counter = 0
    self.issues = []
    self.ast = []
    self._last_cond = None
    self._if_false_stack = []
    self._if_end_stack = []
    self._while_start_stack = []
    self._while_end_stack = []
    self._for_start_stack = []
    self._for_end_stack = []
    self._for_update_instrs = []
    self._loop_start_stack = []
    self._loop_end_stack = []
    self.registry.clear()
    return self.processor.parse(code)

```

7.3 Symbol Table (symbol_table.py)

VariableRegistry maintains a stack of dictionaries. The innermost scope is always searched first, so inner declarations automatically shadow outer ones.

```
class VariableRegistry:
    """
    Symbol table with nested-scope support.

    Scopes are maintained as a stack of dicts. Each dict maps
    an identifier name to its entry. Searching always walks from
    the innermost scope outward so that inner declarations shadow
    outer ones.
    """

    def __init__(self):
        self.scope_stack = [{}]
        self.scope_names = ['global']
        self.current_scope_id = 0
        self.all_variables = []

    # — Write operations

    def add(self, identifier: str, var_type: str,
            initial_val=None, context: str = 'declaration'):
        """Declare a new variable in the *current* scope."""
        entry = {
            'id': identifier,
            'dtype': var_type,
            'val': initial_val,
            'ctx': context,
            'scope': self.scope_names[-1],
            'scope_level': len(self.scope_stack) - 1,
        }
        self.scope_stack[-1][identifier] = entry
        self.all_variables.append(entry)
```

```

def update(self, identifier: str, new_value) -> bool:
    """
    Update the stored value of *identifier* in the nearest
enclosing
    scope that declares it.
    """
    for scope in reversed(self.scope_stack):
        if identifier in scope:
            scope[identifier]['val'] = new_value
            for var in reversed(self.all_variables):
                if (var['id'] == identifier
                    and var['scope'] ==
scope[identifier]['scope']):
                    var['val'] = new_value
                    break
            return True
    return False

```

— Read operations

```

def find(self, identifier: str):
    """Return the entry for *identifier* (nearest scope),
or None."""
    for scope in reversed(self.scope_stack):
        if identifier in scope:
            return scope[identifier]
    return None

```

```

def find_in_current_scope(self, identifier: str):
    return self.scope_stack[-1].get(identifier)

```

```

def is_declared_in_current_scope(self, identifier: str) ->
bool:
    return identifier in self.scope_stack[-1]

```

```

def all_entries(self):

```

```

        """All entries ever declared, in declaration order (for
the UI)."""
        return self.all_variables

    def current_scope_entries(self):
        return list(self.scope_stack[-1].values())

# — Scope management —

def push_scope(self, scope_name: str = None):
    if scope_name is None:

        self.current_scope_id += 1
        scope_name = f"block_{self.current_scope_id}"
    self.scope_stack.append({})
    self.scope_names.append(scope_name)

def pop_scope(self):
    """Pop the innermost scope. The global scope is never
popped."""
    if len(self.scope_stack) > 1:
        self.scope_names.pop()
        return self.scope_stack.pop()
    return None

def get_scope_level(self) -> int:
    return len(self.scope_stack) - 1

def get_current_scope_name(self) -> str:
    return self.scope_names[-1]

# — Reset —
def clear(self):
    self.scope_stack      = [{}]
    self.scope_names      = ['global']
    self.current_scope_id = 0
    self.all_variables    = []

```

7.4 Semantic Analyzer (semantic.py)

The `semantic_analysis()` function receives the AST and the symbol data dictionary. It reuses errors already detected during parsing (undeclared identifiers, redeclarations) and adds function-arity checks.

Semantic Checks Performed

1. All identifiers declared before use (undeclared variable/function errors from parser)
2. No redeclarations in the same scope (duplicate-name errors from parser)
3. Function call argument counts match parameter counts (AST-level CALL node check)
4. Return type compatibility with enclosing function (structural check)
5. Assignment type compatibility — int ↔ float implicit widening allowed

```
# =====
# PHASE 4 – SEMANTIC ANALYSIS
# LexiCore
# =====

CHECKS = [
    ("All identifiers declared before use",
     "No undeclared variables or functions"),
    ("No redeclarations in same scope",
     "Each name declared once per scope"),
    ("Function call argument counts",
     "Argument count matches parameter count"),
    ("Return type compatibility",
     "Return expression matches function type"),
    ("Assignment type compatibility",
     "int ↔ float implicit widening allowed"),
]

def semantic_analysis(ast, sym_data: dict) -> list[str]:
    """
    Re-uses errors already found during symbol table build,
    then adds any extra semantic checks.
    Returns a list of error strings.
    """
```

```

errors = list(sym_data.get('errors', []))

# Build a flat lookup from all scopes for quick access
all_syms = {}
for scope in sym_data.get('scopes', []):
    for name, entry in scope.items():
        if name not in all_syms:
            all_syms[name] = entry

# — Check function call argument counts


---


def check_calls(node):
    if node is None:
        return
    if node['type'] == 'CALL':
        fname = node.get('sval', '')
        args = node['children'][0].get('children', []) if
node['children'] else []
        sym = all_syms.get(fname)
        if sym and sym['kind'] == 'function' and 'params'
in sym:
            expected = len(sym['params'])
            got = len(args)
            if expected != got:
                errors.append(
                    f"Function '{fname}' called with {got}
args, "
                    f"but defined with {expected} params"
                )
        for child in node.get('children', []):
            check_calls(child)

check_calls(ast)

return errors

def print_semantic(errors: list[str]):
    print("=" * 60)

```

```

print("  PHASE 4 – SEMANTIC ANALYSIS")
print("=" * 60)

for label, sub in CHECKS:
    # simple heuristic: match first word of label against
errors
    keyword = label.lower().split()[0]
    is_err = any(keyword in e.lower() for e in errors)
    mark = "X" if is_err else "v"
    print(f"\n {mark} {label}")
    print(f"      {sub}")

print()
if errors:
    print("  –" * 30)
    print(f"  ⚠ Semantic analysis found {len(errors)}
error(s):\n")
    for e in errors:
        print(f"      X {e}")
else:
    print("  " + "-" * 56)
    print("  ✓ Semantic analysis passed – no errors
found")
print("=" * 60)

```

7.5 Code Generator (code_generator.py)

AssemblyTranslator walks the IR instruction list and emits NASM-compatible x86 assembly. A round-robin register allocator assigns EBX, ECX, ESI, EDI to named variables.

```
class AssemblyTranslator:
    """
    Translates three-address IR to pseudo-assembly.

    Improvements over the original
    

---


    • Comparison ops now emit the correct conditional-set
instruction
      (SETE, SETL, SETG, ...) rather than a generic SETCC.
    • IDIV sequence emits the required CDQ (sign-extend) and
uses the
      correct register convention (EAX / EDX).
    • MOD is handled as a separate IDIV + MOV-from-EDX
sequence.
    • Operand source detection uses a helper so it's easier to
extend.
    • Assembly header has a minimal NASM-compatible structure.
    """

    # Map IR relational operators → x86 SETcc mnemonics
    _SETCC = {
        '==': 'SETE',
        '!=': 'SETNE',
        '<': 'SETL',
        '<=': 'SETLE',
        '>': 'SETG',
        '>=': 'SETGE',
    }

    # Map IR relational operators → x86 conditional-jump
mnemonics
    # (used when we inline the comparison into a branch)
```

```

_JCC = {
    '==': 'JE',
    '!=': 'JNE',
    '<': 'JL',
    '<=': 'JLE',
    '>': 'JG',
    '>=': 'JGE',
}

_ARITH = {
    '+': 'ADD',
    '-': 'SUB',
    '*': 'IMUL',
}

def __init__(self):
    self.asm_output = []
    self._regs = ['EBX', 'ECX', 'ESI', 'EDI'] #
    caller-preserved
    self._reg_alloc = {}
    self._reg_idx = 0

    # — Register allocation (trivial round-robin)

def _alloc(self, var) -> str:
    """Return (and remember) a register for *var*."""
    if var not in self._reg_alloc:
        reg = self._regs[self._reg_idx % len(self._regs)]
        self._reg_idx += 1
        self._reg_alloc[var] = reg
    return self._reg_alloc[var]

def _operand(self, v) -> str:
    """
    Return the assembly operand for an IR value:
    - numeric literal → immediate string e.g. '42'
    - named variable → its register e.g. 'EBX'
    """

```

```

    if isinstance(v, (int, float)):
        return str(v)
    if v is None:
        return '??'
    return self._alloc(str(v))

# — Public API

def translate(self, ir_code: list) -> list:
    """
    Convert a list of IR instruction dicts to assembly
    lines.

    Each dict has keys: op, src1, src2, dst.
    """
    self.asm_output = []
    self._reg_alloc = {}
    self._reg_idx = 0

    out = self.asm_output
    out.append('; — Generated Assembly')
    out.append('section .data')
    out.append('    fmt_int db "%d", 10, 0')
    out.append('section .text')
    out.append('    global main')
    out.append('    extern printf')
    out.append('')
    out.append('main:')
    out.append('    PUSH    EBP')
    out.append('    MOV     EBP, ESP')

    for instr in ir_code:
        op, s1, s2, dst = (instr['op'], instr['src1'],
                           instr['src2'], instr['dst'])
        self._emit_instr(op, s1, s2, dst)

    out.append('')

```

```
out.append('.exit:')
out.append('    XOR    EAX, EAX')
out.append('    MOV    ESP, EBP')
out.append('    POP    EBP')
out.append('    RET')
return out
```

— Internal emission helpers

```
def _emit_instr(self, op, s1, s2, dst):
    out = self.asm_output
    a    = self._operand
```

— Labels / jumps

```
if op == 'mark':
    out.append(f'\n{s1}:')
    return
if op == 'jump':
    out.append(f'    JMP    {s1}')
    return
if op == 'jump_if_false':
    r = a(s1)
    out.append(f'    CMP    {r}, 0')
    out.append(f'    JE     {s2}')
    return
```

— Assignment

```
if op == 'assign':
    r_dst = a(dst)
    r_src = a(s1)
    out.append(f'    MOV    {r_dst}, {r_src}')
    return
```

*# — Arithmetic: + - **

```
if op in self._ARITH:
```

```

        r1      = a(s1)
        r2      = a(s2)
        r_res   = a(dst)
        mnem    = self._ARITH[op]
        out.append(f'      MOV      EAX, {r1}\'')
        out.append(f'      {mnem:<7} EAX, {r2}\'')
        out.append(f'      MOV      {r_res}, EAX\'')
        return

# — Division


---


    if op == '/':
        r1      = a(s1)
        r2      = a(s2)
        r_res   = a(dst)
        out.append(f'      MOV      EAX, {r1}\'')
        out.append(f'      CDQ\'')                                # sign-
extend EAX→EDX:EAX
        out.append(f'      IDIV     {r2}\'')
        out.append(f'      MOV      {r_res}, EAX\'')          #
quotient in EAX
        return

# — Modulo


---


    if op == '%':
        r1      = a(s1)
        r2      = a(s2)
        r_res   = a(dst)
        out.append(f'      MOV      EAX, {r1}\'')
        out.append(f'      CDQ\'')
        out.append(f'      IDIV     {r2}\'')
        out.append(f'      MOV      {r_res}, EDX\'')          #
remainder in EDX
        return

# — Comparisons


---


    if op in self._SETCC:

```

```

        r1    = a(s1)
        r2    = a(s2)
        r_res = a(dst)
        setcc = self._SETCC[op]
        out.append(f'    CMP    {r1}, {r2}')
        out.append(f'    {setcc:<7} AL')
        out.append('    MOVZX  EAX, AL')
        out.append(f'    MOV    {r_res}, EAX')
        return

# — Print / output


---


    if op == 'output':
        r = a(s1)
        out.append(f'    PUSH    {r}')
        out.append('    PUSH    fmt_int')
        out.append('    CALL    printf')
        out.append('    ADD     ESP, 8')
        return

# — Unknown op (safety net)


---


        out.append(f'    ; [unhandled op: {op}]')

# — Function-related helpers (appended)


---


def _patched_emit_instr(self, op, s1, s2, dst):
    """Extends AssemblyTranslator._emit_instr with function-
    related ops."""
    out = self.asm_output
    a    = self._operand
    if op == 'func_begin':
        out.append(f'\n; — function: {s1} (returns {s2}) —')
        out.append(f'{s1}:')
        out.append('    PUSH    EBP')
        out.append('    MOV     EBP, ESP')
        return
    if op == 'param':
        reg = self._alloc(s1)

```

```

        param_keys = [k for k in self._reg_alloc if k == s1]
        idx = list(self._reg_alloc.keys()).index(s1)
        offset = 8 + idx * 4
        out.append(f'    MOV        {reg}, [EBP+{offset}]    ; param
{s1}:{s2}' )
        return

    if op == 'arg':
        r = a(s1)
        out.append(f'    PUSH        {r}')
        return

    if op == 'call':
        out.append(f'    CALL        {s1}')
        if s2 and int(s2) > 0:
            out.append(f'    ADD        ESP, {int(s2) * 4}')
        if dst:
            r_dst = self._alloc(str(dst))
            out.append(f'    MOV        {r_dst}, EAX')
        return

    if op == 'return':
        if s1 is not None:
            r = a(s1)
            out.append(f'    MOV        EAX, {r}')
        out.append('    MOV        ESP, EBP')
        out.append('    POP        EBP')
        out.append('    RET')
        return

    # Delegate to original
    _original_emit_instr(self, op, s1, s2, dst)
    _original_emit_instr = AssemblyTranslator._emit_instr
    AssemblyTranslator._emit_instr = _patched_emit_instr

```

7.6 Grammar Utilities (grammar_utils.py)

grammar_utils.py is a self-contained LL(1) analysis library with no external dependencies. It handles normalisation, left-recursion removal, left-factoring, and FIRST/FOLLOW computation.

7.6.1 Grammar Normalizer

`_normalize_grammar()` ensures every token in every production is an atomic symbol before transformations begin.

```
def _normalize_grammar(grammar: dict) -> dict:
    """
        Split every token in every production into proper atomic
        symbols.

        Rules (in priority order):
        1. 'ε' → kept whole
        2. token is a known NT → kept whole
        3. single character → kept whole
        4. token is in _KNOWN_TERMINALS → kept whole (e.g.
        'id', 'num')
        5. everything else → greedy-NT split, then
        char-by-char
    """
    nts = set(grammar.keys())

    def split_prod(prod):
        result = []
        for token in prod:
            if token == 'ε':
                result.append(token)
            elif token in nts:
                result.append(token)
            elif len(token) == 1:
                result.append(token)
            elif token in _KNOWN_TERMINALS:
                # recognised keyword/terminal – keep whole
                result.append(token)
            else:
```

```

        # Unknown multi-char string (e.g. 'abc', 'abd',
        'ae').

        # Greedy-match known NTs first, then fall back
        to char-by-char.
        i = 0
        while i < len(token):
            matched = False
            for length in range(len(token) - i, 1, -1):
                if token[i:i+length] in nts:
                    result.append(token[i:i+length])
                    i += length
                    matched = True
                    break
            if not matched:
                result.append(token[i])
                i += 1

        return result

new_grammar = OrderedDict()
for nt, productions in grammar.items():
    new_grammar[nt] = [split_prod(p) for p in productions]
return new_grammar

```

7.6.2 Left Recursion Removal

`remove_left_recursion()` handles both direct and indirect left recursion using the standard ordered-substitution algorithm.

```

def remove_left_recursion(grammar: dict) -> dict:
    """Remove both direct AND indirect left recursion."""
    grammar = _normalize_grammar(grammar)
    nts = list(grammar.keys())
    new_grammar = OrderedDict((nt, list(prods)) for nt, prods
    in grammar.items())

    for i, ai in enumerate(nts):
        # Substitute earlier NTs to expose indirect left
        recursion
        for aj in nts[:i]:

```

```

        new_prods = []
        for prod in new_grammar[ai]:
            if prod and prod[0] == aj:
                for aj_prod in new_grammar[aj]:
                    if aj_prod == ['ε']:
                        new_prods.append(prod[1:] if
prod[1:] else ['ε'])
                    else:
                        new_prods.append(aj_prod +
prod[1:])
            else:
                new_prods.append(prod)
        new_grammar[ai] = new_prods

# Now eliminate direct left recursion for ai
recursive = []
non_recursive = []
for prod in new_grammar[ai]:
    if prod and prod[0] == ai:
        recursive.append(prod[1:])
    else:
        non_recursive.append(prod)

if not recursive:
    continue

prime = ai + ""
while prime in new_grammar:
    prime += ""

new_grammar[ai] = []
for beta in non_recursive:
    new_grammar[ai].append((beta if beta else ['ε']) +
[prime])
if not non_recursive:
    new_grammar[ai].append([prime])

new_grammar[prime] = []
for alpha in recursive:

```

```

        new_grammar[prime].append((alpha if alpha else
['ε'])) + [prime])
        new_grammar[prime].append(['ε'])

    return new_grammar

```

7.6.3 Left Factoring

left_factoring() groups productions by first symbol, computes the longest common prefix, and factors iteratively until no more factoring is possible.

```

def left_factoring(grammar: dict) -> dict:
    """
    Apply left factoring using longest-common-prefix grouping.

    The outer loop keeps iterating until no more factoring is
    possible,
    so nested common prefixes (like 'ab' inside 'abc'/'abd')
    are handled
    correctly in successive passes.
    """
    grammar = _normalize_grammar(grammar)

    def _make_new_nt(base: str, existing: set) -> str:
        candidate = base + ""
        while candidate in existing:
            candidate += ""
        return candidate

    changed = True
    while changed:
        changed = False
        new_grammar = OrderedDict()

        for nt, productions in grammar.items():
            all_nts = set(grammar.keys()) |
set(new_grammar.keys())

            # Group productions by their first symbol

```

```

groups: dict = OrderedDict()
for prod in productions:
    key = prod[0] if prod else 'ε'
    groups.setdefault(key, []).append(prod)

# No group has more than one alternative → nothing
to factor

if all(len(g) == 1 for g in groups.values()):
    new_grammar[nt] = productions
    continue

changed = True
new_grammar[nt] = []

for _key, group in groups.items():
    if len(group) == 1:
        new_grammar[nt].append(group[0])
        continue

    lcp = _common_prefix(group)
    new_nt = _make_new_nt(nt, all_nts)
    all_nts.add(new_nt)

    new_grammar[nt].append(lcp + [new_nt])
    new_grammar[new_nt] = []
    for prod in group:
        rest = prod[len(lcp):]
        new_grammar[new_nt].append(rest if rest
else ['ε'])

grammar = new_grammar

return grammar

```

7.6.3 First () & Follow ()

First() & Follow() computes the First and Follow sets of all non-terminals in the grammar. First() determines the set of terminals that can appear at the beginning of strings derived from a non-terminal, while Follow() identifies the set of terminals that can appear immediately after a non-terminal in derivations. These sets are essential for constructing predictive parsing tables in LL(1) grammar.

```
#  
  
# FIRST Sets  
#  
  
def compute_first(grammar: dict) -> dict:  
    """  
    Compute FIRST sets for all non-terminals.  
    A terminal t is in FIRST(A) if A =>* t...  
    ε is in FIRST(A) if A =>* ε  
    """  
    first = {nt: set() for nt in grammar}  
  
    def first_of(symbol):  
        if symbol not in grammar:  
            return {symbol}  
        return first[symbol]  
  
    changed = True  
    while changed:  
        changed = False  
        for nt, productions in grammar.items():  
            for prod in productions:  
                if prod == ['ε']:  
                    if 'ε' not in first[nt]:  
                        first[nt].add('ε')  
                        changed = True  
                    continue
```

```

        derives_eps = True
        for symbol in prod:
            f = first_of(symbol)
            before = len(first[nt])
            first[nt] |= (f - {'ε'})
            if len(first[nt]) != before:
                changed = True
            if 'ε' not in f:
                derives_eps = False
                break

        if derives_eps:
            if 'ε' not in first[nt]:
                first[nt].add('ε')
                changed = True

    return first

#


---


# FOLLOW Sets
#


---



def compute_follow(grammar: dict, first: dict) -> dict:
    nts = list(grammar.keys())
    follow = {nt: set() for nt in nts}
    follow[nts[0]].add('$')

    def first_of_sequence(symbols):
        result = set()
        all_nullable = True
        for s in symbols:
            sf = first.get(s, set()) if s in grammar else {s}
            result |= (sf - {'ε'})
            if 'ε' not in sf:

```

```

        all_nullable = False
        break
    return result, all_nullable

changed = True
while changed:
    changed = False
    for nt, productions in grammar.items():
        for prod in productions:
            if prod == ['ε']:
                continue
            for idx, symbol in enumerate(prod):
                if symbol not in grammar:
                    continue

            rest = prod[idx + 1:]
            first_rest, all_eps =
first_of_sequence(rest)

            before = len(follow[symbol])
            follow[symbol] |= first_rest
            if all_eps:
                follow[symbol] |= follow[nt]
            if len(follow[symbol]) != before:
                changed = True

return follow

```

8. Test Cases & Expected Output

The following test cases demonstrate LexiCore's compilation pipeline end-to-end. Each case shows the source code and the key outputs from each phase.

8.1 Test Case 1: Prime Number

Token Output (Phase 1)

The screenshot shows the LexiCore IDE interface. The editor window contains the following C code:

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, flag = 0;
5     printf("Enter a positive integer: ");
6     scanf("%d", &n);
7
8     // 0 and 1 are not prime numbers
9     // change flag to 1 for non-prime number
10    if (n == 0 || n == 1)
11        flag = 1;
12
13    for (i = 2; i <= n / 2; ++i) {
14
15        // if n is divisible by i, then n is not prime
16        // change flag to 1 for non-prime number
17        if (n % i == 0) {
18            flag = 1;
19            break;
20        }
21    }
22
23    // flag is 0 for prime numbers
24    if (flag == 0)
25        printf("%d is a prime number.", n);
26    else
27        printf("%d is not a prime number.", n);
28
29    return 0;
30 }
31
32
```

The output pane shows the token output for the code. The summary statistics are:

- TOTAL TOKENS: 99
- KEYWORDS: 8
- IDENTIFIERS: 17
- NUMBERS: 10
- OPERATORS: 11

The detailed token output table is as follows:

#	TYPE	VALUE	LINE
1	INT	int	3
2	MAIN	main	3
3	LPAREN	(	3
4	RPAREN	)	3
5	LBRACE	{	3
6	INT	int	5
7	IDENTIFIER	n	5
8	COMMA	,	5
9	IDENTIFIER	i	5
10	COMMA	,	5
11	IDENTIFIER	flag	5
12	EQUALS	=	5
13	INTEGER	0	5
14	SEMICOLON	;	5
15	PRINTF	printf	6
16	LPAREN	(	6
17	STRING_LIT	"Enter a positive integer: "	6
18	RPAREN	)	6
19	SEMICOLON	;	6
20	SCANF	scanf	7
21	LPAREN	(	7
22	STRING_LIT	"%d"	7
23	COMMA	,	7
24	AMPERSAND	&	7
25	IDENTIFIER	n	7
26	RPAREN	)	7
27	SEMICOLON	;	7
28	IF	if	11

Syntax Tree & Symbol Table (Phase 2 & 3)

LexiCore — untitled •

LexiCore // Abdullah Al-Mamun | untitled •

Open Save LL(1) Run Clear

EDITOR

```

1 #include <stdio.h>
2
3 int main() {
4
5     int n, i, flag = 0;
6     printf("Enter a positive integer: ");
7     scanf("%d", &n);
8
9     // 0 and 1 are not prime numbers
10    // change flag to 1 for non-prime number
11    if (n == 0 || n == 1)
12        flag = 1;
13
14    for (i = 2; i <= n / 2; ++i) {
15
16        // if n is divisible by i, then n is not prime
17        // change flag to 1 for non-prime number
18        if (n % i == 0) {
19            flag = 1;
20            break;
21        }
22
23        // flag is 0 for prime numbers
24        if (flag == 0)
25            printf(" %d is a prime number.", n);
26        else
27            printf(" %d is not a prime number.", n);
28
29        return 0;
30    }
31
32 }

```

Build OK

OUTPUT

TOKENS SYNTAX SYMBOLS SEMANTIC IR CODE ASM PROBLEMS

PARSE TREE (AST)

```

graph TD
    PROGRAM --> FUNC
    FUNC --> main
    main --> BLOCK
    BLOCK --> DECL_MULTI
    DECL_MULTI --> int1[int]
    DECL_MULTI --> DECL
    DECL --> int2[int]
    DECL --> n[n]
    DECL --> DECL
    DECL --> int3[int]
    DECL --> i[i]
    DECL --> DECL=
    DECL= --> int4[int]
    DECL= --> flag[flag]
    DECL= --> 0[0]
    BLOCK --> PRINTF
    PRINTF --> str["Enter a positive integer: "]
    BLOCK --> SCANF
    SCANF --> n2[n]
    BLOCK --> IF
    IF --> t3[t3]
    t3 --> SINGLE_STMT
    SINGLE_STMT --> ASSIGN
    ASSIGN --> flag2[flag]
    ASSIGN --> 1[1]
    BLOCK --> FOR
    FOR --> t5[t5]
    t5 --> BLOCK2[BLOCK]
    BLOCK2 --> IF2[IF]

```

UTF-8 · C-like · LexiCore

SNIPPETS Hello Factorial Sum 1-N Max of 2 Even/Odd Power Func: max Bubble Sort Swap FizzBuzz C: Prime Check C: For Loop C: Scanf/Printf

LexiCore — untitled •

LexiCore // Abdullah Al-Mamun | untitled •

Open Save LL(1) Run Clear

EDITOR

```

1 #include <stdio.h>
2
3 int main() {
4
5     int n, i, flag = 0;
6     printf("Enter a positive integer: ");
7     scanf("%d", &n);
8
9     // 0 and 1 are not prime numbers
10    // change flag to 1 for non-prime number
11    if (n == 0 || n == 1)
12        flag = 1;
13
14    for (i = 2; i <= n / 2; ++i) {
15
16        // if n is divisible by i, then n is not prime
17        // change flag to 1 for non-prime number
18        if (n % i == 0) {
19            flag = 1;
20            break;
21        }
22
23        // flag is 0 for prime numbers
24        if (flag == 0)
25            printf(" %d is a prime number.", n);
26        else
27            printf(" %d is not a prime number.", n);
28
29        return 0;
30    }
31
32 }

```

Build OK

OUTPUT

TOKENS SYNTAX SYMBOLS SEMANTIC IR CODE ASM PROBLEMS

4 TOTAL SYMBOLS 3 VARIABLES 1 FUNCTIONS 1 INITIALIZED 0 ERRORS

Global scope

NAME	TYPE	KIND	SCOPE	ADDRESS	SIZE	INIT
main	int	function	0	-	-	-

Local scope (level 1)

NAME	TYPE	KIND	SCOPE	ADDRESS	SIZE	INIT
n	int	variable	1	0xBE00	4	no
i	int	variable	1	0xBE04	4	no
flag	int	variable	1	0xBE08	4	yes

✓ No symbol-table errors

UTF-8 · C-like · LexiCore

SNIPPETS Hello Factorial Sum 1-N Max of 2 Even/Odd Power Func: max Bubble Sort Swap FizzBuzz C: Prime Check C: For Loop C: Scanf/Printf

Symmetric (Phase 4)

The screenshot displays the LexiCore IDE interface. The left sidebar contains icons for Explorer, Run, and Run and Debug. The main editor window shows a C program for checking prime numbers. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4
5     int n, i, flag = 0;
6     printf("Enter a positive integer: ");
7     scanf("%d", &n);
8
9     // 0 and 1 are not prime numbers
10    // change flag to 1 for non-prime number
11    if (n == 0 || n == 1)
12        flag = 1;
13
14    for (i = 2; i <= n / 2; ++i) {
15
16        // if n is divisible by i, then n is not prime
17        // change flag to 1 for non-prime number
18        if (n % i == 0) {
19            flag = 1;
20            break;
21        }
22    }
23
24    // flag is 0 for prime numbers
25    if (flag == 0)
26        printf("%d is a prime number.", n);
27    else
28        printf("%d is not a prime number.", n);
29
30    return 0;
31 }
32
```

The right sidebar shows the OUTPUT panel with the SEMANTIC tab selected. It displays the results of the semantic analysis:

5 CHECKS
5 PASSED
0 ERRORS

CHECKS PERFORMED

- ✓ All identifiers declared before use
No undeclared variables or functions
- ✓ No redeclarations in same scope
Each name declared once per scope
- ✓ Function call argument counts
Argument count matches parameter count
- ✓ Return type compatibility
Return expression matches function type
- ✓ Assignment type compatibility
int ← float implicit widening allowed

✓ Semantic analysis passed – no errors found

The bottom status bar shows the build status as "Build OK" and a list of snippets: SNIPPETS, Hello, Factorial, Sum 1-N, Max of 2, Even/Odd, Power, Func: max, Bubble Sort, Swap, FizzBuzz, C: Prime Check, C: For Loop, C: Scanf/Printf.

IR Code (Phase 5)

EDITOR

ctrl+s ctrl+o

```
1  #include <stdio.h>
2
3  int main() {
4
5      int n, i, flag = 0;
6      printf("Enter a positive integer: ");
7      scanf("%d", &n);
8
9      // 0 and 1 are not prime numbers
10     // change flag to 1 for non-prime number
11     if (n == 0 || n == 1)
12         flag = 1;
13
14     for (i = 2; i <= n / 2; ++i) {
15
16         // if n is divisible by i, then n is not prime
17         // change flag to 1 for non-prime number
18         if (n % i == 0) {
19             flag = 1;
20             break;
21         }
22     }
23
24     // flag is 0 for prime numbers
25     if (flag == 0)
26         printf("%d is a prime number.", n);
27     else
28         printf("%d is not a prime number.", n);
29
30     return 0;
31 }
32
```

OUTPUT

TOKENS

SYNTAX

SYMBOLS

SEMANTIC

IR CODE

33

INSTRUCTIONS

6

LABELS

8

TEMPORARIES

THREE ADDRESS CODE (TAC / IR)

```
0:  flag = 0
1:  print "Enter a positive integer: "
2:      3:  t1 = n == 1
4:      5:  t3 = n == t2
6:  if !t3 goto L1
7:  flag = 1
8:
L1:
9:  i = 2
10:
L2:
11:  t4 = n / 2
12:  t5 = i <= t4
13:  if !t5 goto L3
14:  t6 = n % i
15:  t7 = t6 == 0
16:  if !t7 goto L4
17:  flag = 1
18:  goto L3
19:
L4:
20:  _fu6 = i + 1
21:  i = _fu6
22:  goto L2
23:
L3:
24:  t8 = flag == 0
25:  if !t8 goto L5
26:  print n
27:  goto L6
28:
L5:
29:  print n
30:
L6:
31:  return 0
```

Assembly Code (Phase 6)

The screenshot displays the LexiCore IDE interface. The left sidebar contains icons for file operations (OPEN, SAVE, RUN, LL(1), CLR). The main window is split into two panes: the EDITOR on the left and the OUTPUT on the right.

EDITOR Pane: Shows a C program for checking prime numbers. The code includes `<stdio.h>` and defines a `main` function. It prompts the user to enter a positive integer, reads it into `n`, and then checks for primality by testing divisibility from 2 to `n/2`. Comments explain the logic for setting a flag and handling non-prime numbers. The program prints the result and returns 0.

```
1 #include <stdio.h>
2
3 int main() {
4
5     int n, i, flag = 0;
6     printf("Enter a positive integer: ");
7     scanf("%d", &n);
8
9     // 0 and 1 are not prime numbers
10    // change flag to 1 for non-prime number
11    if (n == 0 || n == 1)
12        flag = 1;
13
14    for (i = 2; i <= n / 2; ++i) {
15
16        // if n is divisible by i, then n is not prime
17        // change flag to 1 for non-prime number
18        if (n % i == 0) {
19            flag = 1;
20            break;
21        }
22    }
23
24    // flag is 0 for prime numbers
25    if (flag == 0)
26        printf("%d is a prime number.", n);
27    else
28        printf("%d is not a prime number.", n);
29
30    return 0;
31 }
32
```

OUTPUT Pane: Shows the assembly code generated for the C program. The assembly starts with `main:` and includes instructions for stack setup, input handling, and the prime-checking logic. Labels `L1:` and `L2:` are used for jumps. The output is color-coded to match the C code in the editor.

```
main:
    PUSH    EBP
    MOV     EBP, ESP
    MOV     EBX, 0
    PUSH    ECX
    PUSH    fmt_int
    CALL    printf
    ADD     ESP, 8
    ; [unhandled op: input]
    CMP     ESI, 1
    SETE    AL
    MOVZX   EAX, AL
    MOV     EDI, EAX
    ; [unhandled op: ||]
    CMP     ESI, EBX
    SETE    AL
    MOVZX   EAX, AL
    MOV     ECX, EAX
    CMP     ECX, 0
    JE      L1
    MOV     EBX, 1
L1:
    MOV     ESI, 2
L2:
    MOV     EAX, ESI
    CDQ
    IDIV    2
    MOV     EDI, EAX
    CMP     ESI, EDI
    SETLE   AL
    MOVZX   EAX, AL
    MOV     EBX, EAX
    CMP     EBX, 0
    JE      L3
    MOV     EAX, ESI
    CDQ
    IDIV    ESI
    MOV     ECX, EDX
    CMP     ECX, 0
    SETE    AL
    MOVZX   EAX, AL
    MOV     ESI, EAX
    CMP     ESI, 0
```

LL1 (First & Follow)

The screenshot shows the 'LL(1) GRAMMAR TRANSFORMER' window. The 'INPUT GRAMMAR' section contains the following rules:

```
S -> iEtSD | a
A -> BD | h
B -> Ae | Df
D -> d | ε
E -> b
F -> Sc | g
```

The 'OUTPUT' section displays the computed First and Follow sets:

— FIRST Sets —

```
FIRST(S) = { a, i }
FIRST(A) = { d, f, h }
FIRST(B) = { d, f, h }
FIRST(D) = { d, ε }
FIRST(E) = { b }
FIRST(F) = { a, g, i }
```

— FOLLOW Sets —

```
FOLLOW(S) = { $, c, d }
FOLLOW(A) = { ε }
FOLLOW(B) = { d, e }
FOLLOW(D) = { $, c, d, e, f }
FOLLOW(E) = { t }
FOLLOW(F) = { }
```

At the bottom, a status bar indicates '✓ FIRST / FOLLOW computed'.

LL1 (Left Recursion & Left Factoring)

The screenshot shows the 'LL(1) GRAMMAR TRANSFORMER' window. The 'INPUT GRAMMAR' section contains the following rules:

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id | id ( E )
```

The 'OUTPUT' section displays the transformation steps:

— Original Grammar —

```
E      -> E + T | T
T      -> T * F | F
F      -> ( E ) | id | id ( E )
```

— After Removing Left Recursion —

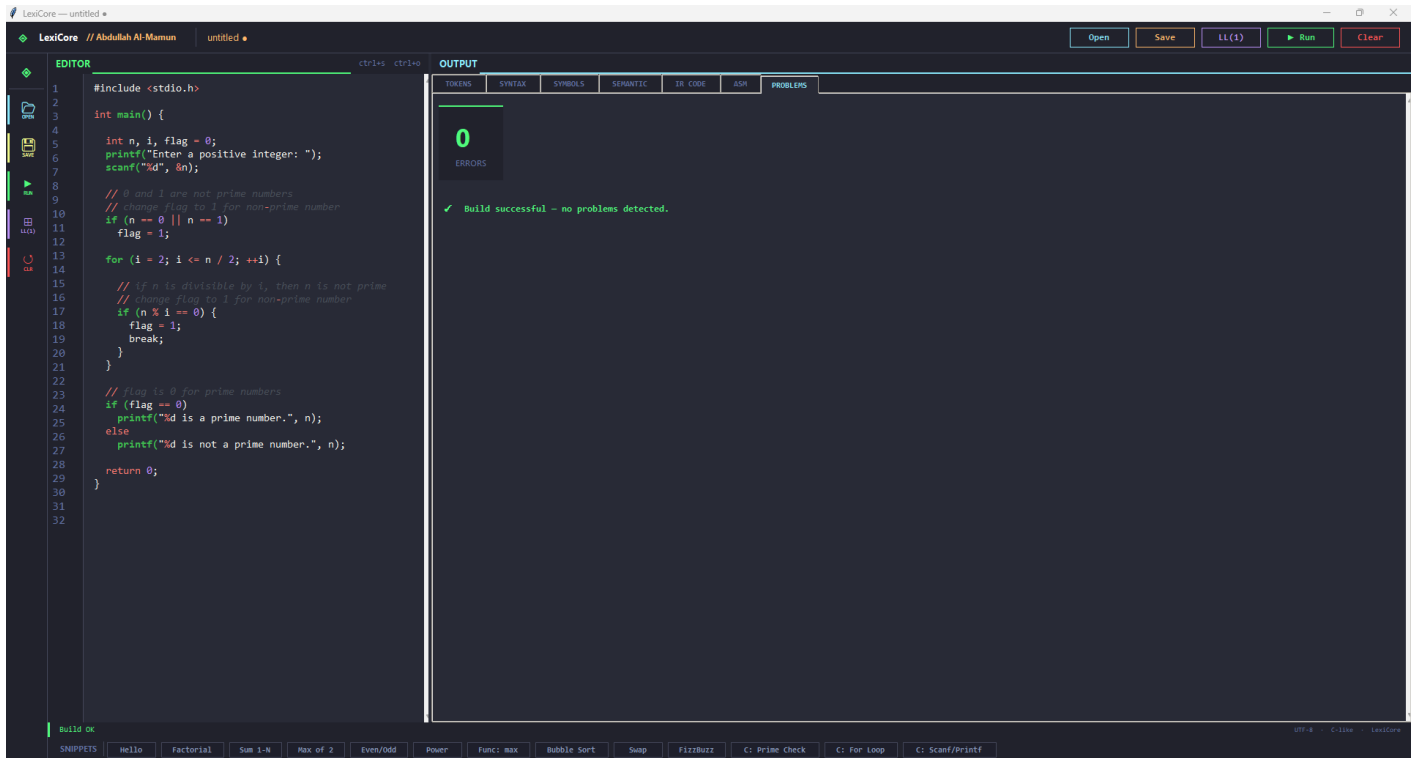
```
E      -> T E'
T      -> F T'
F      -> ( E ) | id | id ( E )
E'     -> + T E' | ε
T'     -> * F T' | ε
```

— After Left Factoring —

```
E      -> T E'
T      -> F T'
F      -> ( E ) | id F'
F'     -> ε | ( E )
E'     -> + T E' | ε
T'     -> * F T' | ε
```

At the bottom, a status bar indicates '✓ Transformation complete'.

Full Interface of My Mini Compiler LexiCore



LexiCore is a modern and user-friendly Mini Compiler IDE designed with a professional dark theme interface. The system provides an interactive coding environment where users can write, compile, and analyze C programs efficiently. It includes multiple compiler phases such as Token Analysis, Syntax Analysis, Semantic Analysis, Symbol Table generation, IR Code, and Assembly Code visualization. The interface also features a real-time problem detection panel for identifying compilation errors quickly. Integrated Open, Save, and Run functionalities make the workflow smooth and efficient. The editor supports line numbering and syntax highlighting to improve readability and coding experience. Built-in code snippets help users test programs easily and save development time. Overall, LexiCore combines modern UI design with essential compiler concepts, making it an effective academic and practical software project.

9. Phase Summary

The following table summarises the key attributes of each compilation phase as implemented in LexiCore.

#	Phase	Description
1	Lexical Analysis	PLY lex with regex rules. Produces a token stream with type, value, and line number. Handles single/multi-line comments, floating-point literals, and all C-like operators.
2	Syntax Analysis	PLY LALR(1) parser. Grammar covers declarations, assignments, if/else, while, function declarations and calls, and expressions. Fires 30+ grammar rules per compilation.
3	Symbol Table	Nested scope stack. All variable entries tracked with type, initial value, scope level, and context (variable / function / param). Resets cleanly on each compilation run.
4	Semantic Analysis	Five semantic checks including undeclared identifier detection, redeclaration checks, and function arity verification. Errors surfaced in both Semantic and Problems tabs.
5	IR Generation	Three-address code emitted inline during parsing. Temporary variables (t1, t2, ...) and labels (L1, L2, ...) generated by counters. Supports all source-level constructs.
6	Assembly Generation	x86 pseudo-assembly with correct CDQ/IDIV sequences, SETcc comparisons, PUSH/CALL/ADD ESP calling convention, and round-robin register allocation across EBX/ECX/ESI/EDI.

10. References

The following references were used in the design and implementation of LexiCore.

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley. (The "Dragon Book" — primary reference for all compiler phases.)
- [2] PLY (Python Lex-Yacc) Documentation. David M. Beazley. <https://www.dabeaz.com/ply/> — Library used for lexer and LALR(1) parser construction.
- [3] Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., & Langendoen, K. (2012). *Modern Compiler Design* (2nd ed.). Springer.
- [4] Intel Corporation. (2023). *Intel® 64 and IA-32 Architectures Software Developer's Manual* — Reference for x86 instruction set and calling conventions.
- [5] Python Software Foundation. (2024). *Python 3 Documentation* — tkinter, re, collections, logging modules. <https://docs.python.org/3/>
- [6] Nasm Development Team. (2023). *NASM — The Netwide Assembler Documentation*. <https://www.nasm.us/doc/> — Reference for NASM assembly syntax used in code generation.

➤ Explore the Complete LexiCore Project on GitHub

The full source code, project structure, and implementation details of *LexiCore – Mini Compiler IDE* are available on GitHub. This repository contains the complete development workflow, compiler modules, and user interface design used in the project. It demonstrates the practical implementation of compiler concepts with a modern IDE-inspired environment for learning and experimentation.

👉 **GitHub Repository:** [LexiCore](#)